Matteo Palmonari
matteo.palmonari@disco.unimib.it

5.2
Information Extraction #1

UNIVERSITÀ DEGLI STUDI DI MILANO BICOCCA
DIPARTIMENTO DI INFORMATICA SISTEMISTICA E COMUNICAZIONE
dat·AI
data * AI
AI * data

Information Extraction from Texts

- Extracting structured information from documents is useful in several downstream applications
  - Data integration
    - Knowledge base population (typically also includes relations)
    - Knoweledge base construction as integration of unstructured data sources (entity-related annotations may be enough)
  - Querying and searching
    - Sematic search
      - e.g., search ”artificial intelligence” in news mentioning Apple Inc.
    - RAG and conversational search interfaces
      - e.g., what is the *number of employees* of Apple Inc?
    - Structured queries
      - e.g., list *CEOs of tech companies* most covered in news
  - Analytics
      - E.g., Compare media coverage for Apple Inc, Google and Meta

Information Extraction: Outline

- Focus of our lessons on IE
  - Some preliminaries about linguistic analysis
  - Named Entity Recognition +++
  - Named Entity Linking ++
  - Relation Extraction +
  - Some more insights (next lesson):
    - More examples of downstream applications
    - Information extraction on tabular data
    - Use cases in the legal domain in Italian (= not EN)

Outline - parts 1 and 2

- **Some preliminaries about linguistic analysis**
- Named Entity Recognition
- Named Entity Linking
- Relation Extraction

spaCy: NLP library for Python
www.spacy.io

Industrial-Strength Natural Language Processing

IN PYTHON

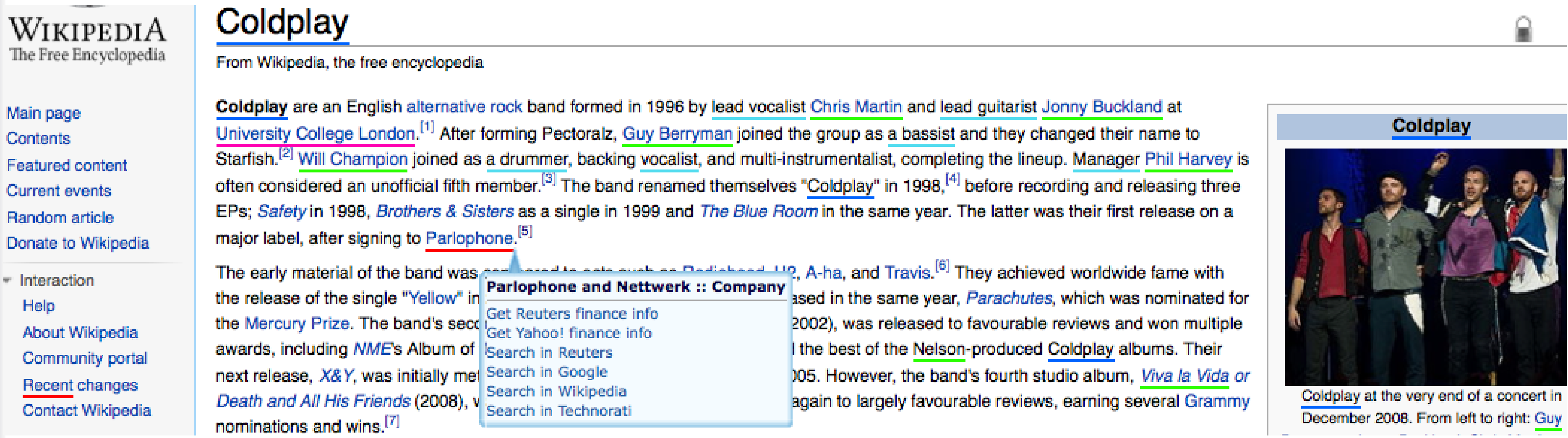
spaCy ✸ **Out now:** spaCy v3.0 USAGE MODELS API UNIVERSE 20,328 Search docs

Features

- Support for **64+ languages**
- **55 trained pipelines** for 17 languages
- Multi-task learning with pretrained **transformers** like BERT
- Pretrained **word vectors**
- State-of-the-art speed
- Production-ready **training system**
- Linguistically-motivated **tokenization**
- Components for **named entity** recognition, part-of-speech tagging, dependency parsing, sentence segmentation, **text classification**, lemmatization, morphological analysis, entity linking and more
- Easily extensible with **custom components** and attributes
- Support for custom models in **PyTorch**, **TensorFlow** and other frameworks
- Built in **visualizers** for syntax and NER
- Easy **model packaging**, deployment and workflow management
- Robust, rigorously evaluated accuracy

Off-the-shelves NER algorithms + rules
(a few more details later)

Named Entity Recognition & Linking



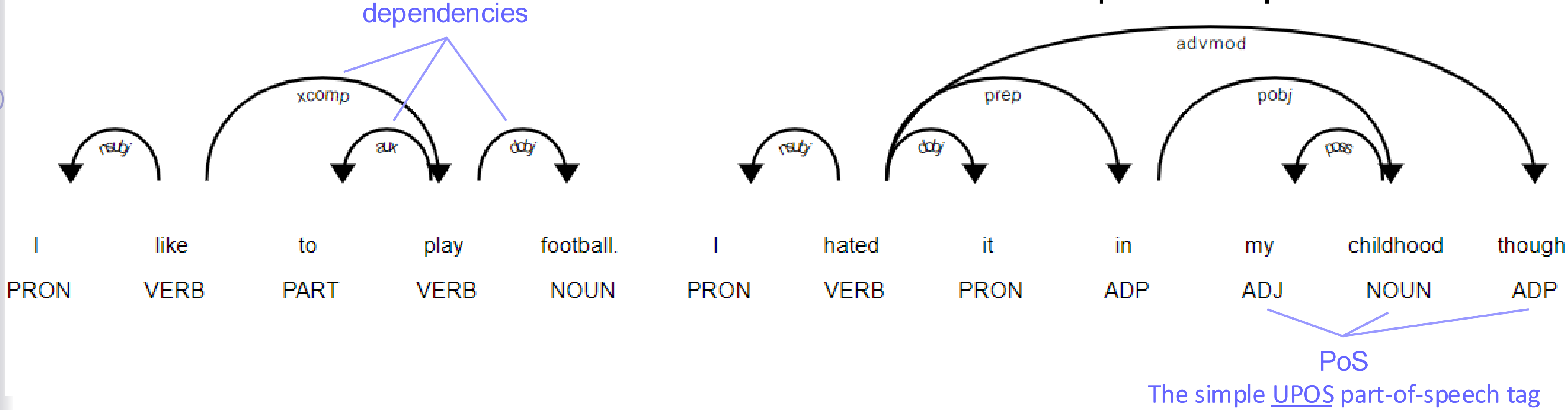
(Clear Forest Gnosis Mozilla Plugin)

- Thought experiment:
  - define an algorithm to identify entities based on what you learned in school
- Hints:
  - Think about grammar …
  - Why “coldplay” and “University College London” but not “are”, “formed”, etc. ?

Part-of-Speech Tagging & Dependency Parsing

Part-of-Speech tagging:
assign a *tag* to each word that specifies its grammatical category

Dependency parsing:
build a *dependency tree* representing the dependencies between the parts of speech



- More tags are defined for English
- Good introduction at https://spacy.io/usage/linguistic-features/
- Interesting pragmatic tutorial: https://laptrinhx.com/stack-abuse-python-for-nlp-parts-of-speech-tagging-and-named-entity-recognition-3135808087/

Question: why can't we always use PoS and DTs?

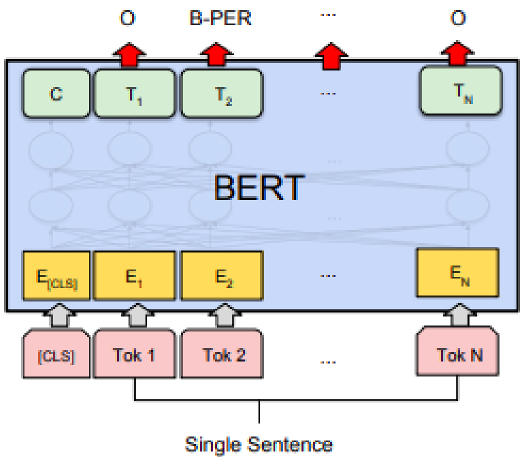
Outline

- Some preliminaries about linguistic analysis
- **Named Entity Recognition**
- Named Entity Linking
- Relation Extraction

Named Entity Recognition

The sentence tagging approach
Idea and BERT for NER

Source: http://jalammar.github.io/illustrated-bert/



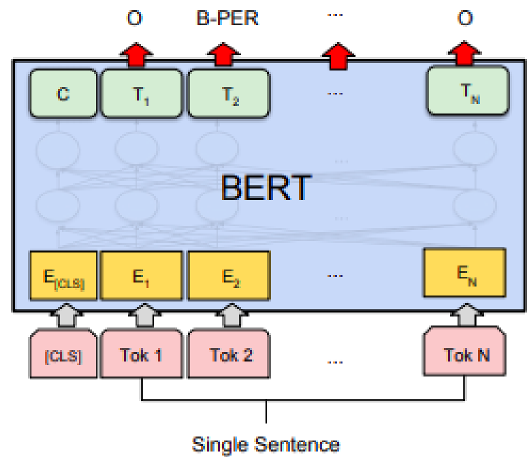
(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

- We exploit the feature extraction capabilities of PLMs
- Sentence tagging: each token of an input sentence is classified in a category
  - In principle, two decisions:
    - out, beginning-of-entity, inside-entity, etc.
    - peson, location, etc.
  - In practice, categories that mix the criteria:
    - out, beginning-of-person, beginning-of-location, etc.
  - Different classification schems can be used, e.g., IOB is the simpler one

- In principle
  - plain fine-tuning of a PLM, e.g., BERT, to perform the tagging task
- In practice
  - combination of processing steps that may include parsing, tagging, etc., as in **spaCy**

The sentence tagging approach
IOB Scheme and Classes

Source: http://jalammar.github.io/illustrated-bert/



(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

IOB SCHEME

- I – Token is **inside** an entity.
- O – Token is **outside** an entity.
- B – Token is the **beginning** of an entity.

BILUO SCHEME

- B – Token is the **beginning** of a multi-token entity.
- I – Token is **inside** a multi-token entity.
- L – Token is the **last** token of a multi-token entity.
- U – Token is a single-token **unit** entity.
- O – Token is **outside** an entity.

CLASSES (EXAMPLES)

- Person
- GPE
- ...

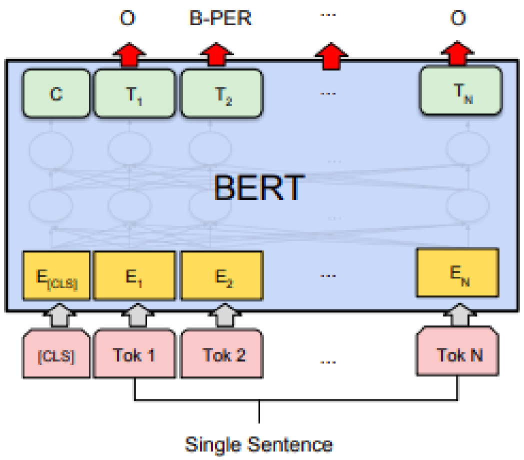
EXAMPLE IN SPACY

TEXT	ENT_IOB	ENT_IOB_	ENT_TYPE_	DESCRIPTION
San	3	B	"GPE"	beginning of an entity
Francisco	1	I	"GPE"	inside an entity
considers	2	O	""	outside an entity
banning	2	O	""	outside an entity
sidewalk	2	O	""	outside an entity
delivery	2	O	""	outside an entity
robots	2	O	""	outside an entity

https://spacy.io/usage/linguistic-features#entity-types

The sentence tagging approach
IOB Scheme and Classes

Source: http://jalammar.github.io/illustrated-bert/



(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

IOB SCHEME
- I – Token is **inside** an entity.
- O – Token is **outside** an entity.
- B – Token is the **beginning** of an entity.

BILUO SCHEME
- B – Token is the **beginning** of a multi-token entity.
- I – Token is **inside** a multi-token entity.
- L – Token is the **last** token of a multi-token entity.
- U – Token is a single-token **unit** entity.
- O – Token is **outside** an entity.

CLASSES (EXAMPLES)
- Person
- GPE
- …

Question:
which limitations if we use an IO scheme (no "B" tags)?

~~ sentence tagging approach

A pipeline in spaCy incl. NER

ROBERTA BASE + SPACY

Label Scheme ¶

The statistical components included in this model package assign the following labels. The labels are specific to the corpus that the model was trained on. To see the description of a label, you can use `spacy.explain`.

TRANSFORMER	
TAGGER	`$`, `''`, `,`, `-LRB-`, `-RRB-`, `.`, `:`, `ADD`, `AFX`, `CC`, `CD`, `DT`, `EX`, `FW`, `HYPH`, `IN`, `JJ`, `JJR`, `JJS`, `LS`, `MD`, `NFP`, `NN`, `NNP`, `NNPS`, `NNS`, `PDT`, `POS`, `PRP`, `PRP$`, `RB`, `RBR`, `RBS`, `RP`, `SYM`, `TO`, `UH`, `VB`, `VBD`, `VBG`, `VBN`, `VBP`, `VBZ`, `WDT`, `WP`, `WP$`, `WRB`, `XX`, ` `` `
PARSER	`ROOT`, `acl`, `acomp`, `advcl`, `advmod`, `agent`, `amod`, `appos`, `attr`, `aux`, `auxpass`, `case`, `cc`, `ccomp`, `compound`, `conj`, `csubj`, `csubjpass`, `dative`, `dep`, `det`, `dobj`, `expl`, `intj`, `mark`, `meta`, `neg`, `nmod`, `npadvmod`, `nsubj`, `nsubjpass`, `nummod`, `oprd`, `parataxis`, `pcomp`, `pobj`, `poss`, `preconj`, `predet`, `prep`, `prt`, `punct`, `quantmod`, `relcl`, `xcomp`
ATTRIBUTE_RULER	
LEMMATIZER	

en_core_web_trf

RELEASE DETAILS

Latest: 3.5.0

English transformer pipeline (roberta-base). Components: transformer, tagger, parser, ner, attribute_ruler, lemmatizer.

LANGUAGE	EN English
TYPE	CORE Vocabulary, syntax, entities
GENRE	WEB written text (blogs, news, comments)
SIZE	TRF 438 MB
COMPONENTS	`transformer`, `tagger`, `parser`, `attribute_ruler`, `lemmatizer`, `ner`
PIPELINE	`transformer`, `tagger`, `parser`, `attribute_ruler`, `lemmatizer`, `ner`
VECTORS	0 keys, 0 unique vectors (0 dimensions)
DOWNLOAD LINK	en_core_web_trf-3.5.0-py3-none-any.whl
SOURCES	OntoNotes 5 (Ralph Weischedel, Martha Palmer, Mitchell Marcus, Eduard Hovy, Sameer Pradhan, Lance Ramshaw, Nianwen Xue, Ann Taylor, Jeff Kaufman, Michelle Franchini, Mohammed El-Bachouti, Robert Belvin, Ann Houston) ClearNLP Constituent-to-Dependency Conversion (Emory University) WordNet 3.0 (Princeton University) roberta-base (Yinhan Liu and Myle Ott and Naman Goyal and Jingfei Du and Mandar Joshi and Danqi Chen and Omer Levy and Mike Lewis and Luke Zettlemoyer and Veselin Stoyanov)
AUTHOR	Explosion
LICENSE	MIT

`ENTS_P`	Named entities (precision)	0.90
`ENTS_R`	Named entities (recall)	0.90
`ENTS_F`	Named entities (F-score)	0.90

NER	`CARDINAL`, `DATE`, `EVENT`, `FAC`, `GPE`, `LANGUAGE`, `LAW`, `LOC`, `MONEY`, `NORP`, `ORDINAL`, `ORG`, `PERCENT`, `PERSON`, `PRODUCT`, `QUANTITY`, `TIME`, `WORK_OF_ART`

https://spacy.io/models/en

~~ sentence tagging approach

A pipeline in spaCy incl. NER

Example of **attribute ruler**: making "The Who" singular proper noun

Editable Code spaCy v3.7 · Python 3 · via Binder

```
import spacy

nlp = spacy.load("en_core_web_sm")
text = "I saw The Who perform. Who did you see?"
doc1 = nlp(text)
print(doc1[2].tag_, doc1[2].pos_)  # DT DET
print(doc1[3].tag_, doc1[3].pos_)  # WP PRON

# Add attribute ruler with exception for "The Who" as NNP/PROPN NNP/PROPN
ruler = nlp.get_pipe("attribute_ruler")
# Pattern to match "The Who"
patterns = [[{"LOWER": "the"}, {"TEXT": "Who"}]]
# The attributes to assign to the matched token
attrs = {"TAG": "NNP", "POS": "PROPN"}
# Add rules to the attribute ruler
ruler.add(patterns=patterns, attrs=attrs, index=0)  # "The" in "The Who"
ruler.add(patterns=patterns, attrs=attrs, index=1)  # "Who" in "The Who"

doc2 = nlp(text)
print(doc2[2].tag_, doc2[2].pos_)  # NNP PROPN
print(doc2[3].tag_, doc2[3].pos_)  # NNP PROPN
# The second "Who" remains unmodified
print(doc2[5].tag_, doc2[5].pos_)  # WP PRON
```

en_core_web_trf

RELEASE DETAILS
Latest: 3.5.0

English transformer pipeline (roberta-base). Components: transformer, tagger, parser, ner, attribute_ruler, lemmatizer.

LANGUAGE	EN English
TYPE	CORE Vocabulary, syntax, entities
GENRE	WEB written text (blogs, news, comments)
SIZE	TRF 438 MB
COMPONENTS	transformer, tagger, parser, attribute_ruler, lemmatizer, ner
PIPELINE	transformer, tagger, parser, attribute_ruler, lemmatizer, ner
VECTORS	0 keys, 0 unique vectors (0 dimensions)
DOWNLOAD LINK	en_core_web_trf-3.5.0-py3-none-any.whl
SOURCES	OntoNotes 5 (Ralph Weischedel, Martha Palmer, Mitchell Marcus, Eduard Hovy, Sameer Pradhan, Lance Ramshaw, Nianwen Xue, Ann Taylor, Jeff Kaufman, Michelle Franchini, Mohammed El-Bachouti, Robert Belvin, Ann Houston) ClearNLP Constituent-to-Dependency Conversion (Emory University) WordNet 3.0 (Princeton University) roberta-base (Yinhan Liu and Myle Ott and Naman Goyal and Jingfei Du and Mandar Joshi and Danqi Chen and Omer Levy and Mike Lewis and Luke Zettlemoyer and Veselin Stoyanov)
AUTHOR	Explosion
LICENSE	MIT

ENTS_P	Named entities (precision)	0.90
ENTS_R	Named entities (recall)	0.90
ENTS_F	Named entities (F-score)	0.90

NER	CARDINAL, DATE, EVENT, FAC, GPE, LANGUAGE, LAW, LOC, MONEY, NORP, ORDINAL, ORG, PERCENT, PERSON, PRODUCT, QUANTITY, TIME, WORK_OF_ART

https://spacy.io/models/en

~~ sentence tagging approach

A pipeline in spaCy incl. NER

ROBERTA BASE + SPACY

Label Scheme ¶

The statistical components included in this model package assign the following labels. The labels are specific to the corpus that the model was trained on. To see the description of a label, you can use `spacy.explain`.

TRANSFORMER	
TAGGER	`$`, `''`, `,`, `-LRB-`, `-RRB-`, `.`, `:`, `ADD`, `AFX`, `CC`, `CD`, `DT`, `EX`, `FW`, `HYPH`, `IN`, `JJ`, `JJR`, `JJS`, `LS`, `MD`, `NFP`, `NN`, `NNP`, `NNPS`, `NNS`, `PDT`, `POS`, `PRP`, `PRP$`, `RB`, `RBR`, `RBS`, `RP`, `SYM`, `TO`, `UH`, `VB`, `VBD`, `VBG`, `VBN`, `VBP`, `VBZ`, `WDT`, `WP`, `WP$`, `WRB`, `XX`, ` `` `
PARSER	`ROOT`, `acl`, `acomp`, `advcl`, `advmod`, `agent`, `amod`, `appos`, `attr`, `aux`, `auxpass`, `case`, `cc`, `ccomp`, `compound`, `conj`, `csubj`, `csubjpass`, `dative`, `dep`, `det`, `dobj`, `expl`, `intj`, `mark`, `meta`, `neg`, `nmod`, `npadvmod`, `nsubj`, `nsubjpass`, `nummod`, `oprd`, `parataxis`, `pcomp`, `pobj`, `poss`, `preconj`, `predet`, `prep`, `prt`, `punct`, `quantmod`, `relcl`, `xcomp`
ATTRIBUTE_RULER	
LEMMATIZER	

en_core_web_trf

RELEASE DETAILS

Latest: `3.5.0`

English transformer pipeline (roberta-base). Components: transformer, tagger, parser, ner, attribute_ruler, lemmatizer.

LANGUAGE	EN English
TYPE	CORE Vocabulary, syntax, entities
GENRE	WEB written text (blogs, news, comments)
SIZE	TRF 438 MB
COMPONENTS	`transformer`, `tagger`, `parser`, `attribute_ruler`, `lemmatizer`, `ner`
PIPELINE	`transformer`, `tagger`, `parser`, `attribute_ruler`, `lemmatizer`, `ner`
VECTORS	0 keys, 0 unique vectors (0 dimensions)
DOWNLOAD LINK	en_core_web_trf-3.5.0-py3-none-any.whl
SOURCES	OntoNotes 5 (Ralph Weischedel, Martha Palmer, Mitchell Marcus, Eduard Hovy, Sameer Pradhan, Lance Ramshaw, Nianwen Xue, Ann Taylor, Jeff Kaufman, Michelle Franchini, Mohammed El-Bachouti, Robert Belvin, Ann Houston) ClearNLP Constituent-to-Dependency Conversion (Emory University) WordNet 3.0 (Princeton University) roberta-base (Yinhan Liu and Myle Ott and Naman Goyal and Jingfei Du and Mandar Joshi and Danqi Chen and Omer Levy and Mike Lewis and Luke Zettlemoyer and Veselin Stoyanov)
AUTHOR	Explosion
LICENSE	MIT

`ENTS_P`	Named entities (precision)	0.90
`ENTS_R`	Named entities (recall)	0.90
`ENTS_F`	Named entities (F-score)	0.90

NER	`CARDINAL`, `DATE`, `EVENT`, `FAC`, `GPE`, `LANGUAGE`, `LAW`, `LOC`, `MONEY`, `NORP`, `ORDINAL`, `ORG`, `PERCENT`, `PERSON`, `PRODUCT`, `QUANTITY`, `TIME`, `WORK_OF_ART`

https://spacy.io/models/en

Why “~~”sentence tagging? Transition-based NER

- Because it is a slightly different model: a **transition-based model**
- Process the input sequence and decide which action to take
- Inspired by [Lample & al.: Neural Architectures for Named Entity Recognition]
    - Actions: **SHIFT** (Buffer > Stack), **REDUCE-label** (Stack > Output), **OUT** (Buffer > Output)
    - In practice **REDUCE-label** corresponds to *n* actions, one for each of the *n* type label
    - After reduction, encode the completed chunk using its tokens plus a learned label embedding
- spaCy’s model: transition-based BILUO system + different features

steps

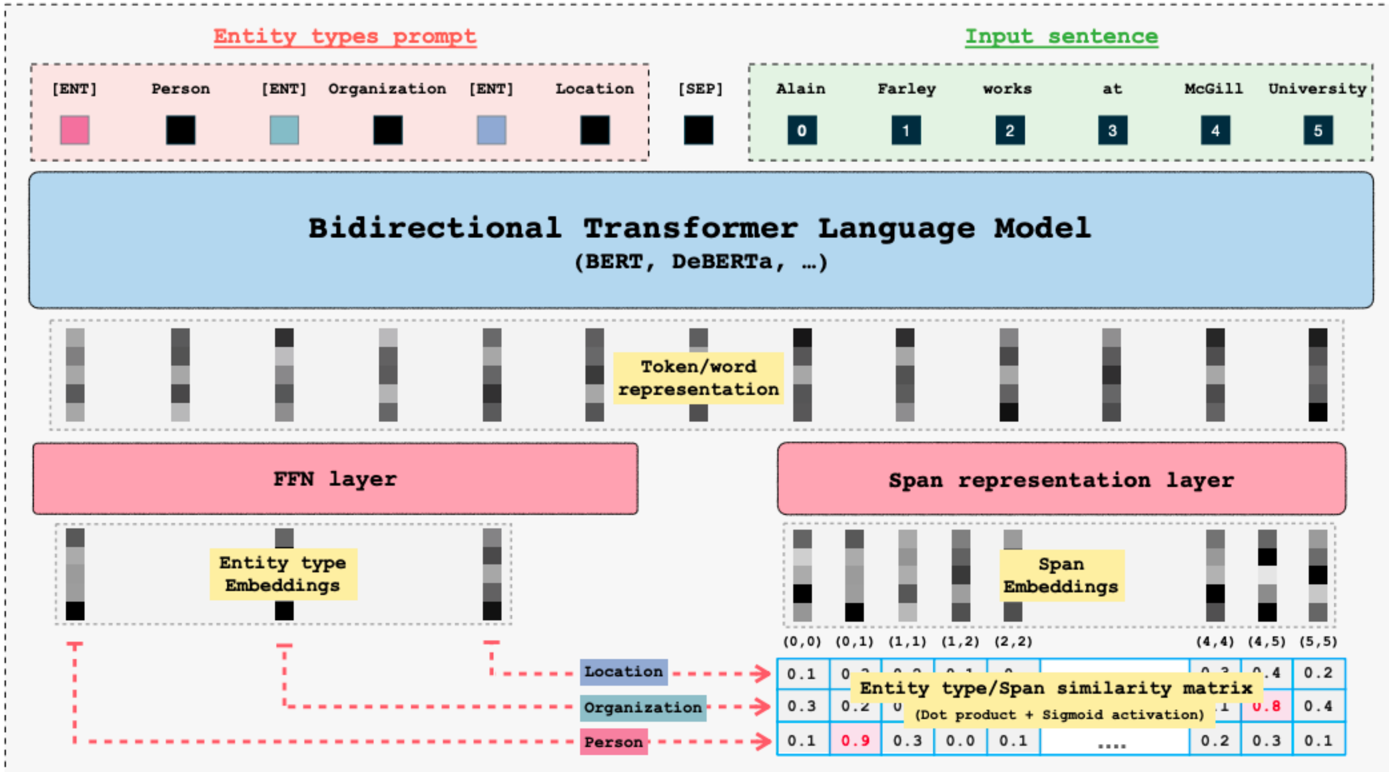
Transition	Output	Stack	Buffer	Segment
	[]	[]	[Mark, Watney, visited, Mars]	
SHIFT	[]	[Mark]	[Watney, visited, Mars]	
SHIFT	[]	[Mark, Watney]	[visited, Mars]	
REDUCE(PER)	[(Mark Watney)-PER]	[]	[visited, Mars]	(Mark Watney)-PER
OUT	[(Mark Watney)-PER, visited]	[]	[Mars]	
SHIFT	[(Mark Watney)-PER, visited]	[Mars]	[]	
REDUCE(LOC)	[(Mark Watney)-PER, visited, (Mars)-LOC]	[]	[]	(Mars)-LOC

Figure 3: Transition sequence for *Mark Watney visited Mars* with the Stack-LSTM model.

NER: Surveys

- History and recent methods
  - See Jurafsky's book
- Recent neural methods
  - Yadav, V., & Bethard, S. (2018, August). A Survey on Recent Advances in Named Entity Recognition from Deep Learning models. In Proceedings of the 27th International Conference on Computational Linguistics (pp. 2145-2158). Arxiv version: https://arxiv.org/abs/1910.11470
  - J. Li, A. Sun, J. Han and C. Li, "**A Survey on Deep Learning for Named Entity Recognition**," in IEEE Transactions on Knowledge and Data Engineering. Arxiv (open) version: https://arxiv.org/abs/1812.09449

GLiNER: Powerful Encoder-based NER



Main idea:
- represent all possible spans as vectors, all type labels as vectors
- compare all spans with all type labels

Intuition:
- we use explicit label embeddings (we can use an arbitrary set of new type labels, by exploiting LMs' representation capabilities)

https://arxiv.org/pdf/2311.08526 * https://aclanthology.org/2025.emnlp-demos.10.pdf

GLiNER: Powerful Encoder-based NER

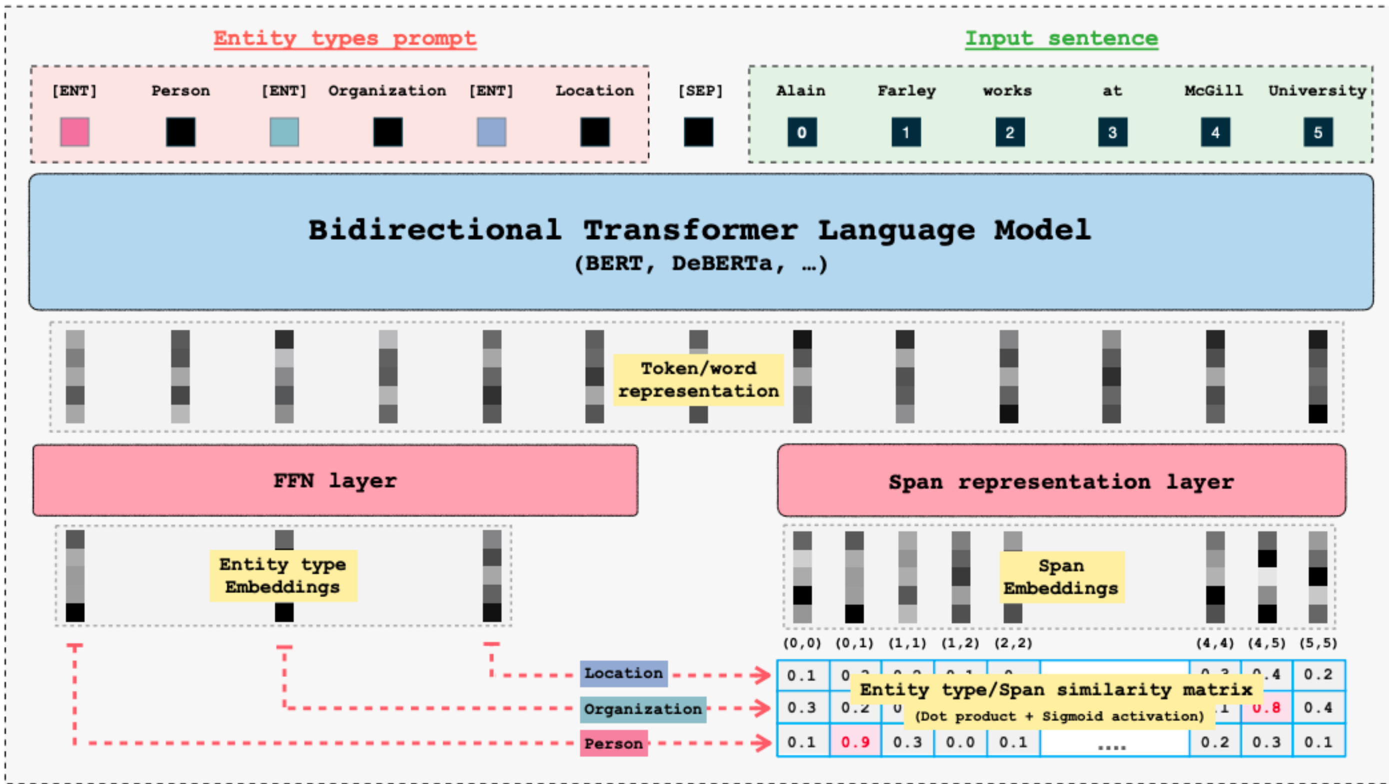


Figure 2: **Model architecture**. GLiNER employs a BiLM and takes as input entity type prompts and a sentence/text. Each entity is separated by a learned token [ENT]. The BiLM outputs representations for each token. Entity embeddings are passed into a FeedForward Network, while input word representations are passed into a span representation layer to compute embeddings for each span. Finally, we compute a matching score between entity representations and span representations (using dot product and sigmoid activation). For instance, in the figure, the span representation of (0, 1), corresponding to "Alain Farley," has a high matching score with the entity embeddings of "Person".

Now GLiNER2* with more tasks

Check also: https://fastino.ai/

https://arxiv.org/pdf/2311.08526 * https://aclanthology.org/2025.emnlp-demos.10.pdf

NER – Latest trends

- Recent neural methods
  - Decompose mention identification and classification in two tasks
  - Transfer learning: from training with a broad set of types to fine-tuning on specific type of interest
  - Generative LLMs
    - **Off-the-shelves xLLMs** or fine-tuned open models like Code-Llama
    - **One** or multi-turn prompts

Input

Defn: An entity is a person(person), university(university), scientist(scientist), ,,,,event(event), award(award), or theory(theory). Abstract scientific concepts can be entities if they have a name associated with them. If an entity does not fit the types above it is (misc). Dates, times, adjectives and verbs are not entities

Q: Given the paragraph below, identify a list of possible entities and for each entry explain why it either is or is not an entity:

Paragraph: He attended the U.S. Air Force Institute of Technology for a year , earning a bachelor 's degree in aeromechanics , and received his test pilot training at Edwards Air Force Base in California before his assignment as a test pilot at Wright-Patterson Air Force Base in Ohio .

Answer:
1. U.S. Air Force Institute of Technology | True | as he attended this institute is likely a university (university)
2. bachelor 's degree | False | as it is not a university, award or any other entity type
3. aeromechanics | True | as it is a scientific discipline (discipline)
4. Edwards Air Force Base | True | as an Air Force Base is an organised unit (organisation)
5. California | True | as in this case California refers to the state of California itself (location)
6. Wright-Patterson Air Force Base | True | as an Air Force Base is an organisation (organisation)
7. Ohio | True | as it is a state (location)

Figure 1: Example of prompt to pre-trained language model. Definition is in blue , question and task in red, example answer and chain of thought format in green

Ashok, D., & Lipton, Z. C. (2023). Promptner: Prompting for named entity recognition. arXiv preprint arXiv:2305.15444.

NER – Latest trends

Input

Defn: An entity is a person(person), university(university), scientist(scientist), ,,,,event(event), award(award), or theory(theory). Abstract scientific concepts can be entities if they have a name associated with them. If an entity does not fit the types above it is (misc). Dates, times, adjectives and verbs are not entities

Q: Given the paragraph below, identify a list of possible entities and for each entry explain why it either is or is not an entity:

Paragraph: He attended the U.S. Air Force Institute of Technology for a year , earning a bachelor 's degree in aeromechanics , and received his test pilot training at Edwards Air Force Base in California before his assignment as a test pilot at Wright-Patterson Air Force Base in Ohio .

Answer:
1. U.S. Air Force Institute of Technology | True | as he attended this institute is likely a university (university)
2. bachelor 's degree | False | as it is not a university, award or any other entity type
3. aeromechanics | True | as it is a scientific discipline (discipline)
4. Edwards Air Force Base | True | as an Air Force Base is an organised unit (organisation)
5. California | True | as in this case California refers to the state of California itself (location)
6. Wright-Patterson Air Force Base | True | as an Air Force Base is an organisation (organisation)
7. Ohio | True | as it is a state (location)

Figure 1: Example of prompt to pre-trained language model. Definition is in blue , question and task in red, example answer and chain of thought format in green

Base for spaCy implementation of LLM-based NER:

https://spacy.io/api/large-language-models#ner

NER – Latest trends

- Recent neural methods
  - Decompose mention identification and classification in two tasks
  - Transfer learning: from training with a broad set of types to fine-tuning on specific type of interest
  - Generative LLMs
    - **Off-the-shelves xLLMs** or fine-tuned open models like Code-Llama
    - One or **multi-turn prompts**

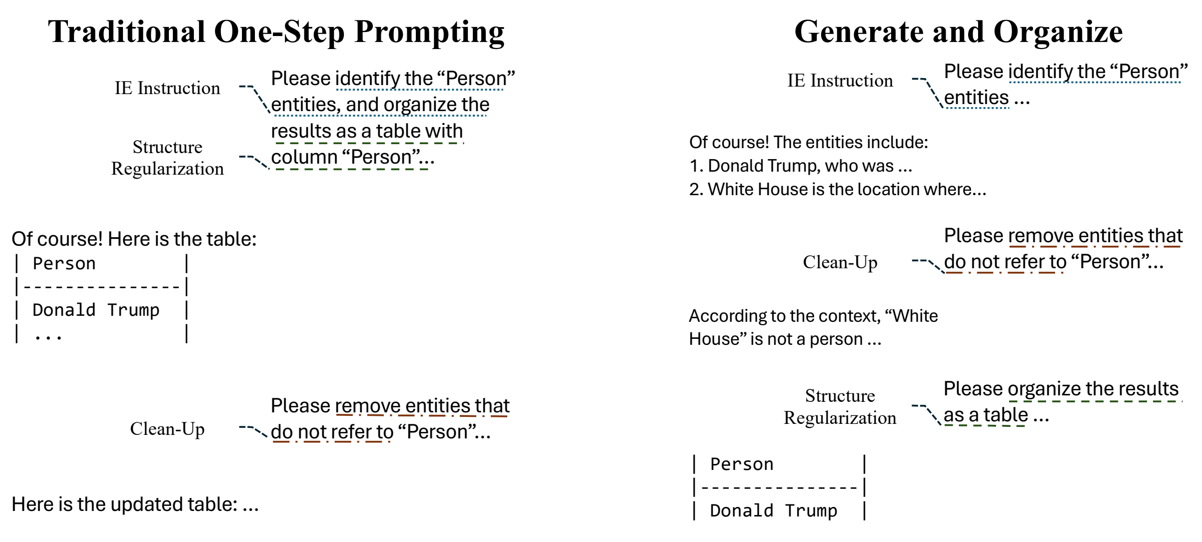


Figure 1: The pipeline of G&O for NER, compared with Traditional One-Step prompting methods.

Li, Y., Ramprasad, R., & Zhang, C. (2024). A Simple but Effective Approach to Improve Structured Language Model Output for Information Extraction. arXiv preprint arXiv:2402.13364.

NER – Latest trends

- Recent neural methods
  - Decompose mention identification and classification in two tasks
  - Transfer learning: from training with a broad set of types to fine-tuning on specific type of interest
  - Generative LLMs
    - Off-the-shelves xLLMs or **fine-tuned open models like Code-Llama**
    - **One** or multi-turn prompts

GOLLIE: Code-LLaMA fine-tuned to follow instructions

SOTA on many datasets with a 34B model
Trained on different IE tasks, tested on NER, Event Extraction and more

Sainz, O., García-Ferrero, I., Agerri, R., de Lacalle, O. L., Rigau, G., & Agirre, E. (2023). Gollie: Annotation guidelines improve zero-shot information-extraction. *arXiv preprint arXiv:2310.03668*.
Also ICLR 24

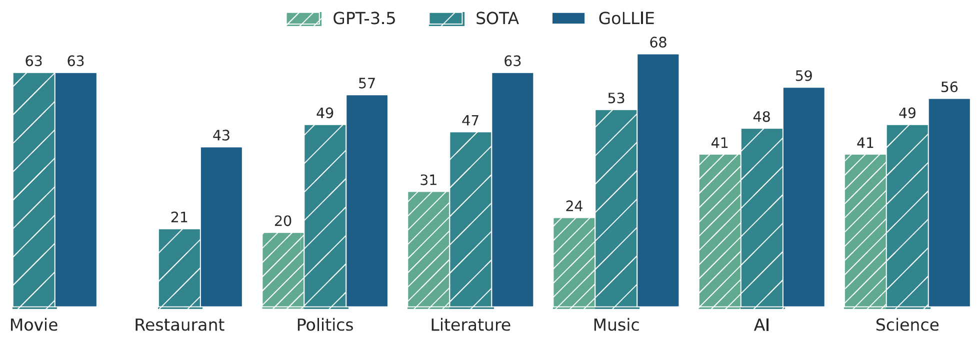


Figure 1: Out of domain zero-shot NER results. GPT results are not available for all domains.

NER – Latest trends

- Recent neural methods
  - Decompose mention identification and classification in two tasks
  - Transfer learning: from training with a broad set of types to fine-tuning on specific type of interest
  - Generative LLMs
    - Off-the-shelves xLLMs or **fine-tuned open models like Code-Llama**
    - **One** or multi-turn prompts

Key idea:
- Teach CodeLLama to follow annotation guidelines (~those given to human annotators)
- Provide examples of strings for an entity class
- Schema for the output

In the example, extraction of **entities** for the classes:
- programming languages
- metrics

GOLLIE: Code-LLaMA fine-tuned to follow instructions

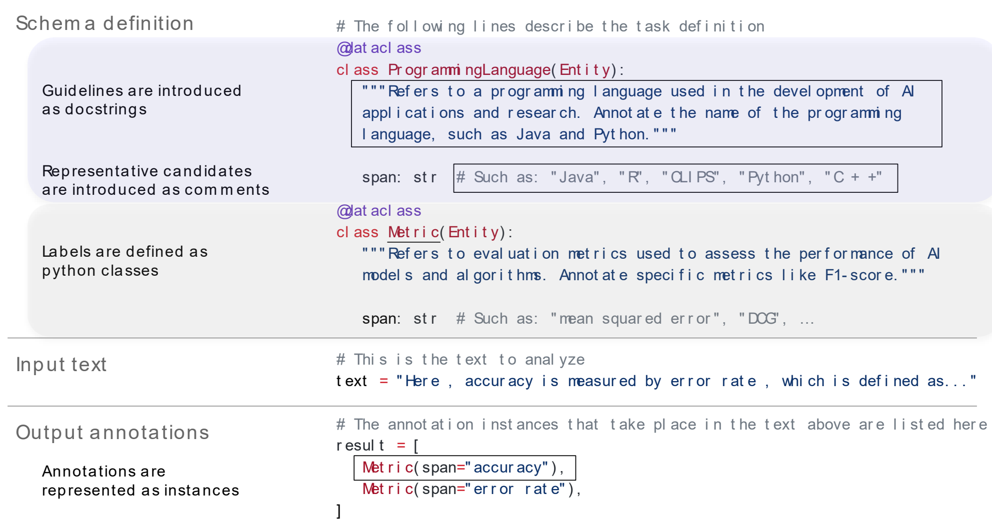


Figure 2: Example of the input and output of the model.

NER

NER – Latest trends

- Recent neural methods
  - Decompose mention identification and classification in two tasks
  - Transfer learning: from training with a broad set of types to fine-tuning on specific type of interest
  - Generative LLMs
    - Off-the-shelves xLLMs or **fine-tuned open models like Code-Llama**
    - **One** or multi-turn prompts

GOLLIE: Code-LLaMA fine-tuned to follow instructions

Key idea:

- Teach CodeLLama to follow annotation guidelines (~those given to human annotators)
- Provide examples of strings for an entity class
- Schema for the output

In the example, extraction of **events** of the class:

- Vulnerability class

Event Extraction

```python
@dataclass
class VulnerabilityPatch(Event):

    mention: str
    cve: List[str]
    issues_addressed: List[str]
    supported_platform: List[str]
    vulnerability: List[str]
    vulnerable_system: List[str]
    releaser: List[str]
    patch: List[str]
    patch_number: List[str]
    system_version: List[str]
    time: List[str]
```

```python
@dataclass
class VulnerabilityPatch(Event):
    """A VulnerabilityPatch Event happens when a software company addresses a
    known vulnerability by releasing or describing an appropriate update."""

    mention: str
    """The text span that triggers the event.
    Such as: patch, fixed, addresses, implemented, released
    """
    cve: List[str] # The vulnerability identifier
    issues_addressed: List[str] # What did the patch fix
    supported_platform: List[str] # The platforms that support the patch
    vulnerability: List[str] # The vulnerability
    vulnerable_system: List[str] # The affected systems
    releaser: List[str] # The entity releasing the patch
    patch: List[str] # What was the patch about
    patch_number: List[str] # Number or name of the patch
    system_version: List[str] # The version of the vulnerable system
    time: List[str] # When was the patch implemented, the date
```

Figure 3: Example of the input representation. (left) An example of an event definition w/o guidelines information. (right) The same example but with guideline information as Python comments.

SKIP

Fine-grained Entity Typing (FET): what is it?

Multi-label, multi-branch → one mention, distinct “specific” types
Annotation can be partial → no leaf types in the annotation
Hierarchy does not cover each possible type → open world assumption

Entity Typing (ET):

- multilabel classification problem
- small type sets (e.g., *Person, Organization, Location*)

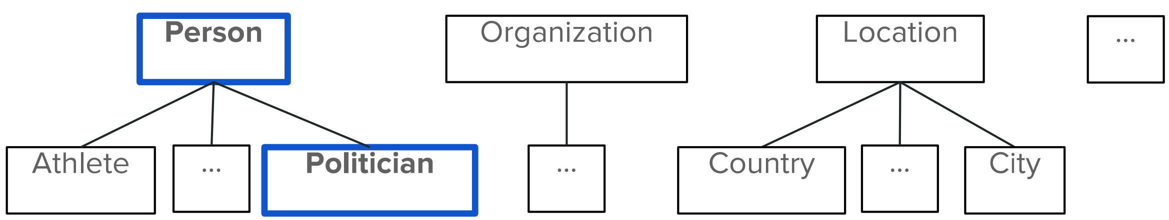
Fine-grained Entity Typing (FET) [Ling & Weld, 2012]**:**

- ET with larger and more complex type sets (e.g., *Person, Athlete, Actor, Politician, Cyclist, …*)
- type sets usually organized hierarchically

Example: *“The former President* ***<Barack Obama>*** *was born in Hawaii”*

left context mention right context

Type Set Hierarchy



SKIP

Zero and Few-shot FET with LLMs

Rubini, R., Vimercati, M., Palmonari, M. **PROMET: Parameter-efficient Few-shot Fine-grained Entity Typing with Implicit Mask Filling**. WI-IAT 2024

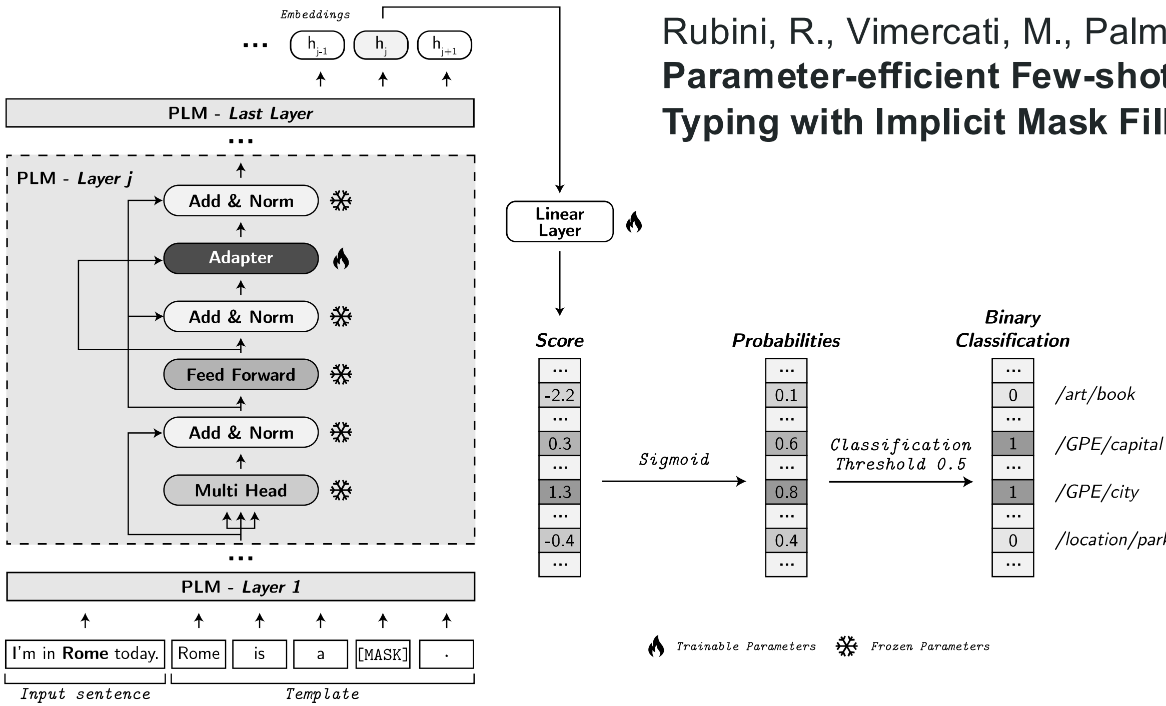


Fig. 2: PROMET architecture: we adapt PLM through adapter modules and use implicit mask filling to obtain label scores. We use *Sigmoid* activation function and a classification threshold to make binary classification in a multi-label scenario.

From Riccardo Rubini's MA-DS thesis presentation
Submitted at The Web Conference

SKIP

Zero and Few-shot FET with LLMs

Table 1: Summary of the dataset used for the evaluation. $|D_{train}|$ and $|D_{dev}|$ are related to the 5-shot samples.

Dataset	Types	Hierarchy	$\|D_{train}\|$	$\|D_{dev}\|$	$\|D_{test}\|$
FEW NERD [11]	74	2 level	330	330	95880
BBN [53]	47	2 level	235	151-154	12349
FIGER [27]	128	2 level	640	308-323	563

< Datasets

Results >

	FEW NERD Macro F1	FEW NERD Micro F1	BBN Macro F1	BBN Micro F1	FIGER Macro F1	FIGER Micro F1
ALIGNIE*(SL + T)	68.10 ±2.53	68.10 ±2.53	-	-	-	-
PROMET$_{(fine-tuned)}$(SL + T)	<u>64.78</u> ±2.59	<u>64.78</u> ±2.59	-	-	-	-
PROMET$_{(adapters)}$(SL + T)	**69.29** ±2.03	**69.29** ±2.03	-	-	-	-
ALIGNIE*(ML)	63.66 ±2.02	63.92 ±2.11	51.01 ±5.05	49.25 ±3.99	70.00 ±3.14	**68.30** ±2.05
PROMET$_{(fine-tuned)}$(ML)	<u>60.00</u> ±2.25	<u>59.65</u> ±2.25	51.69 ±5.00	<u>55.53</u> ±5.06	<u>63.93</u> ±4.88	<u>58.04</u> ±4.05
PROMET$_{(adapters)}$(ML)	<u>**71.02**</u> ±1.43	<u>**70.94**</u> ±1.37	**52.25** ±5.72	<u>55.27</u> ±5.23	**72.23** ±4.39	65.48 ±3.80
ALIGNIE [7]$_{(+ data aug.)}$(SL + T)	69.54	69.54	*uncomparable*		-	-
FiveFine [7](SL + T)	71.88	71.88	*uncomparable*		-	-

	Verbalizer	MLM Head	Adapters	Number of Trainable Parameters *PLM*	*Classifier*	*Total*	Epochs	Emissions
ALIGNIE	✓	✓	✗	125 M	3.72 M	128.72 M	143	0.04 kg CO_2eq
PROMET (fine-tuned)	✗	✗	✗	125 M	0.06 M	125.06 M	44	0.01 kg CO_2eq
PROMET (adapters)	✗	✗	✓	0.89 M	0.06 M	0.95 M	25	0.01 kg CO_2eq

< Resource consumption

NER: Wrap-up

- Named Entity Recognition
  - Identification of entity mentions and classification
  - Widespread downstream usage in multiple applications and domains (legal, finance, news, bio, etc.)
- Identification vs. classification
  - Jointly (typical setup) or in different steps (some recent methods)
- Mention classification
  - From a few types to fine-grained types in deep type taxonomies
- Evolution/different approaches
  - From sequential tagging (classification of tokens in the input text) to generative models
- What model to use?
  - Accuracy not enough
  - Efficiency (what scale of processing?), flexibility and training requirements (how much training data? Can it work in zero-shot settings?), number of labels (how many types?)